Fingerprint Getting Startded Guide

Fingerprint Getting Startded Guide
- 1.Introduction
- 2.SDK Contents
- 3.Development Guide
  - 3.1 Configure your project
    - 3.1.1 Create a new Android project
    - 3.1.2 Enable Java 8
    - 3.1.3 Add aar file
    - 3.1.4 Congratulations
  - 3.2 Sample Code
    - 3.2.1 SM-9X Driver API
    - 3.2.2 Justouch Api
  - 3.3 How to get high-quality fingerprints
  - 3.4 Development reference
    - 3.4.1 Host (USB/SPI communication)
    - 3.4.1 Host (Serial communication)
    - 3.4.1 Module
- 4. Support Contact Information:

1.Introduction

Welcome to use SM-9X SDK. This document will introduce how to use SM-9X SDK for development in Android.

2.SDK Contents

The SDK contains libraries and demo needed for the development of SM-92M-S,Contains the following directories:

- /apk or /bin

  The `apk` or `bin` folder contains the compiled binary files, which can be installed directly on the Android device.

- /demo

  The `demo` folder is the source code corresponding to the apk file, which can be opened using Android Studio .

> NOTE:
>
> The library (.so & .jar & .aar) file is included in the demo and is not provided separately.

3.Development Guide

3.1 Configure your project

This lesson shows you how to create a new Android project using Android Studio and introduces some of the files in the project.

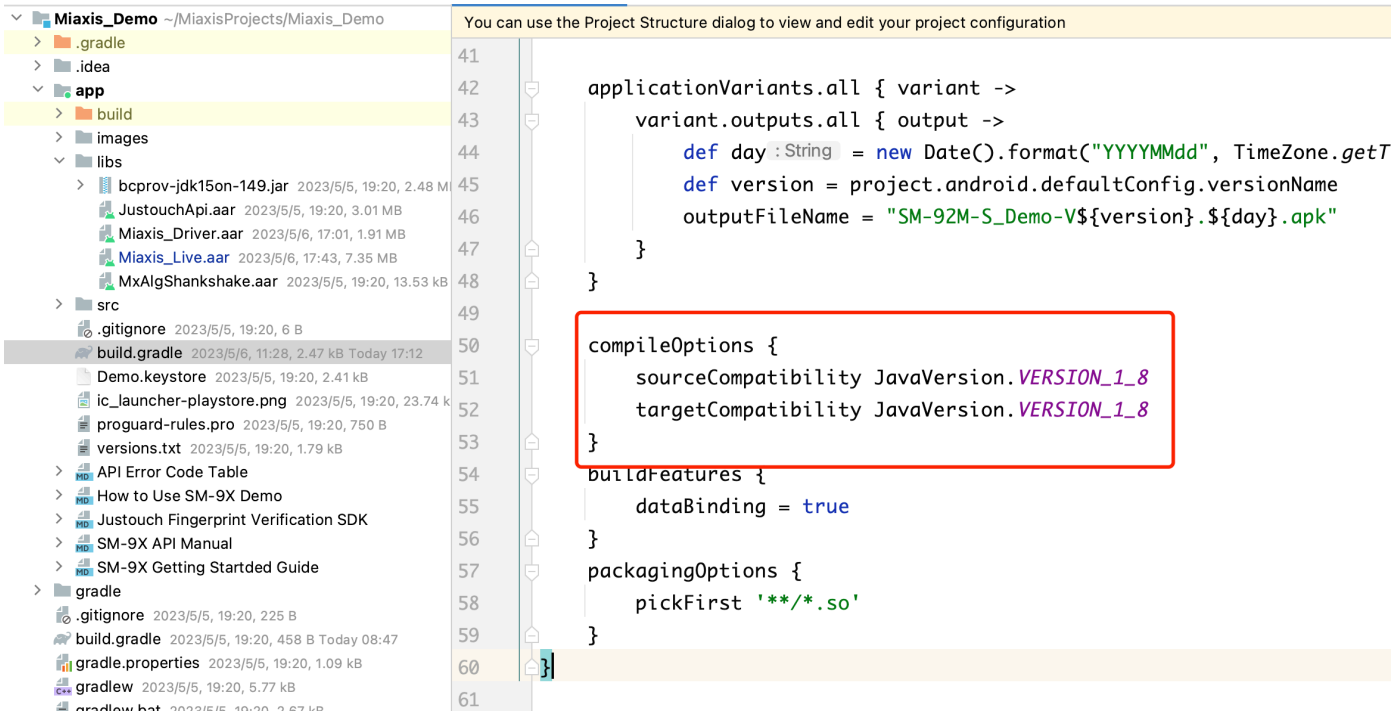
To create a new Android project, follow these steps:

3.1.1 Create a new Android project

Create a new Empty Activity application project,You can refer to [Android Developer]() to create an Android project.

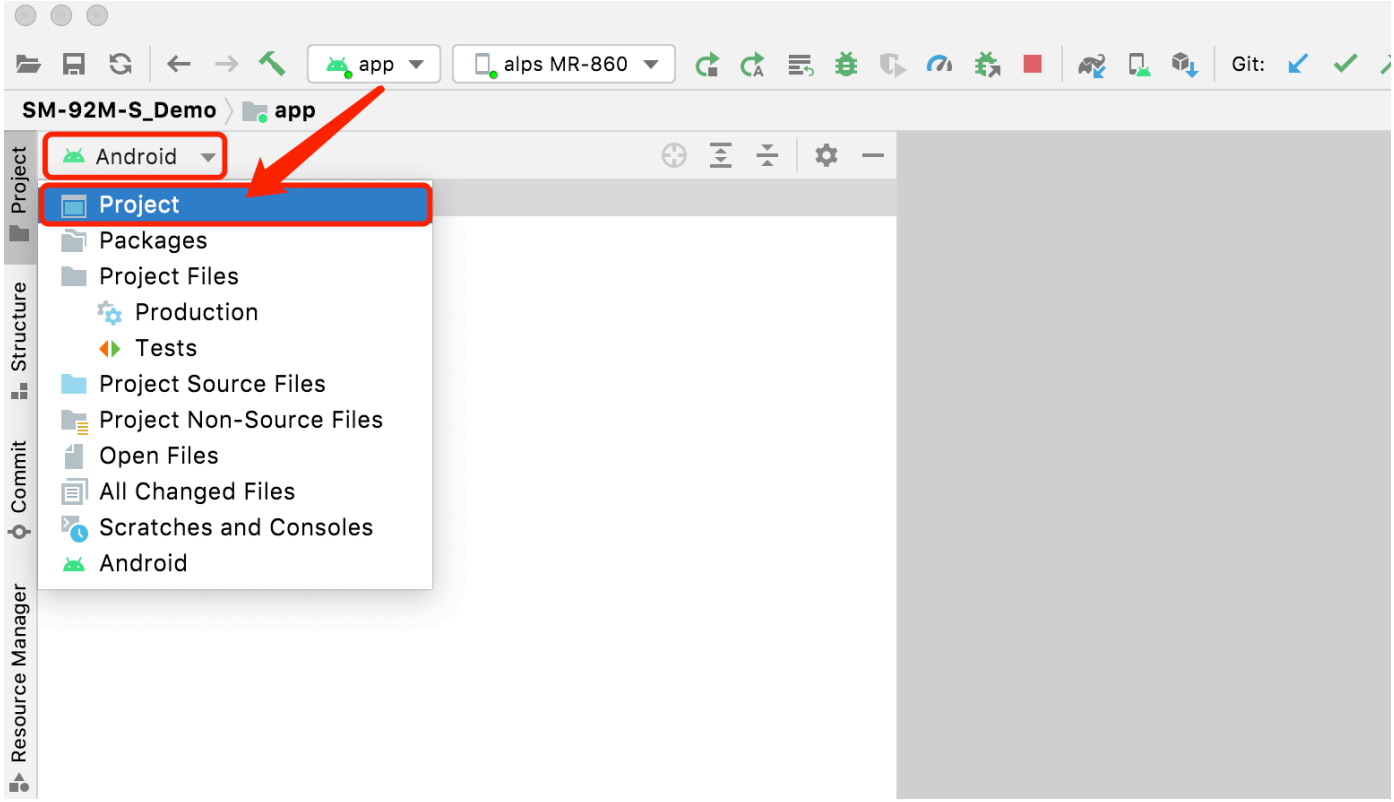
3.1.2 Enable Java 8

The SDK needs to work with **JDK 1.8**, open **build.gradle**, add a configuration inside, as shown in the figure:

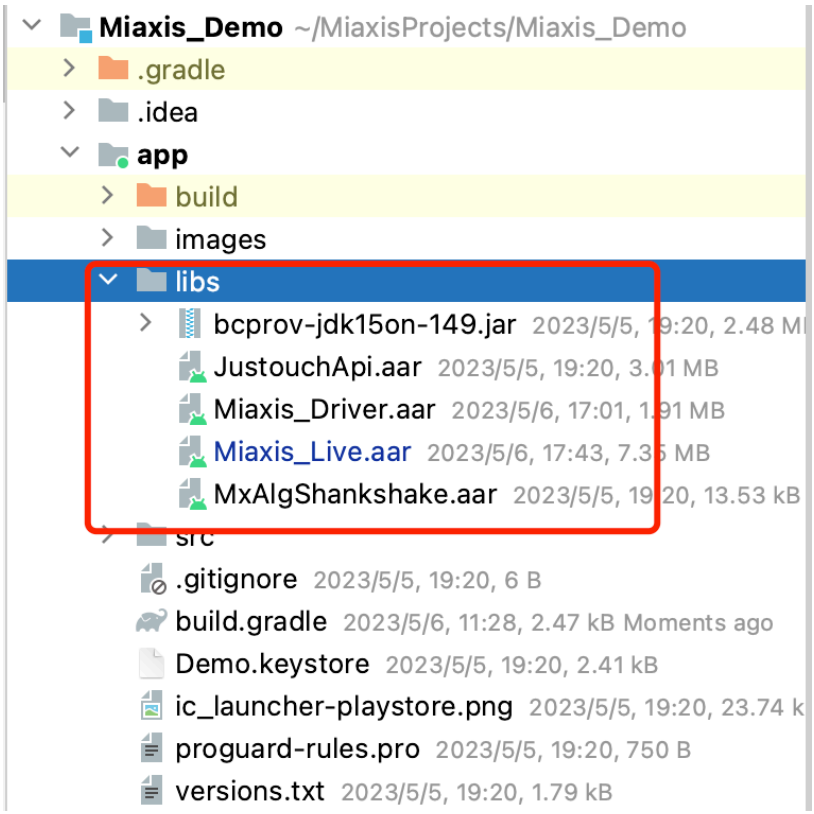


3.1.3 Add aar file

First, switch to the project view. Click on `Android` and select `Project`.As shown below:



Then, copy the aar from the demo project to the lib directory of your project, as shown below:



> NOTE:
>
> `JustouchApi.aar` is used for fingerprint algorithm.
>
> `SM92M-SDriverApi.aar` is used to access `SM-92M-S` Device.

3.1.4 Congratulations

Congratulations, you have completed all the preparations, then you can refer to the **Sample Code** to learn how to use our API.

3.2 Sample Code

3.2.1 SM-9X Driver API

Get a instance of SM-9X driver API :

```
SM92MSVcApi mDriverApi = FingerApiFactory.getInstance(getApplicationContext(),
FingerApiFactory.USB);
// do something with mDriverApi
```

Open/Close SM-92M-S device :

```
int fd = mDriverApi.openDevice();
if (fd >= 0) {
  // Open successfully !
  // 0 means it has been opened before
} else {
  // process error code
}
```

Capture fingerprint image from SM-92X device:

```
CaptureConfig captureConfig = new CaptureConfig.Builder()
                .setLfdLevel(0)
                .setLatentLevel(0)
                .setTimeout(8000)
                .setAreaScore(45)
                .setPreviewCallBack(previewCallBack)
                // AES/ECB/PKCS5Padding
                //.setAESConfig(new
AESConfig.Builder().setKey("1234567890123456").build())
                //.setAESStatus(CaptureConfig.AES_HOST)
                .build();
MxResult<MxImage> image = mDriverApi.getImage(captureConfig);
if (imageResult.isSuccess()) {
  // Capture successfully !
  MxImage mxImage = imageResult.data;
  // Convert raw image to Android bitmap
  byte[] imageDate = new byte[mxImage.width * mxImage.height + 1078];
  BmpLoader.Raw2Bmp(imageDate, mxImage.data, mxImage.width, mxImage.height);
  Bitmap bitmap = BitmapFactory.decodeByteArray(imageDate, 0, imageDate.length);
  //show bitmap
  ImageView imageView ;//The imageView to show fingerprint image
```

```
  imageView.setImageBitmap(bitmap);
} else {
  //process error code
}
```

See the **SM-9X API Manual.pdf** for more driver apis.

3.2.2 Justouch Api

Create a instance of Jutouch Api :

```
JustouchFingerAPI mJustouchApi = new JustouchFingerAPI();
```

Create FMR :

```
MxImage image ; // capture from SM-92M-S
byte[] fmrBuffer = new byte[1024];//Must be 1024 bytes
int result = mJustouchApi.createTemplateISO(image.data, image.width, image.height,
fmrBuffer);
if (result >= 0) {
  // successfully
}
// If you use ISO2011, please call the following function :
// mJustouchApi.createTemplateISO2011(...)
// If you use ANSI, please call the following function :
// mJustouchApi.createTemplateANSI(...)
```

Match two FMR :

```
byte[] fmrBufferA //Must be 1024 bytes
byte[] fmrBufferB //Must be 1024 bytes
int similarScore = mJustouchApi.compareTemplatesANSI(fmrBufferA, fmrBufferB);
if (score >= 45) { // Suggest 45 pass
  // Match passed
} else if (score >= 0) {
  // Match not passed
} else {
  // Process error code
}
```

> NOTE:
>
> When Justouch works with SM-9X, there is no need to call initialization

See the **Justouch Fingerprint Verification SDK.pdf** for more Justouch apis.

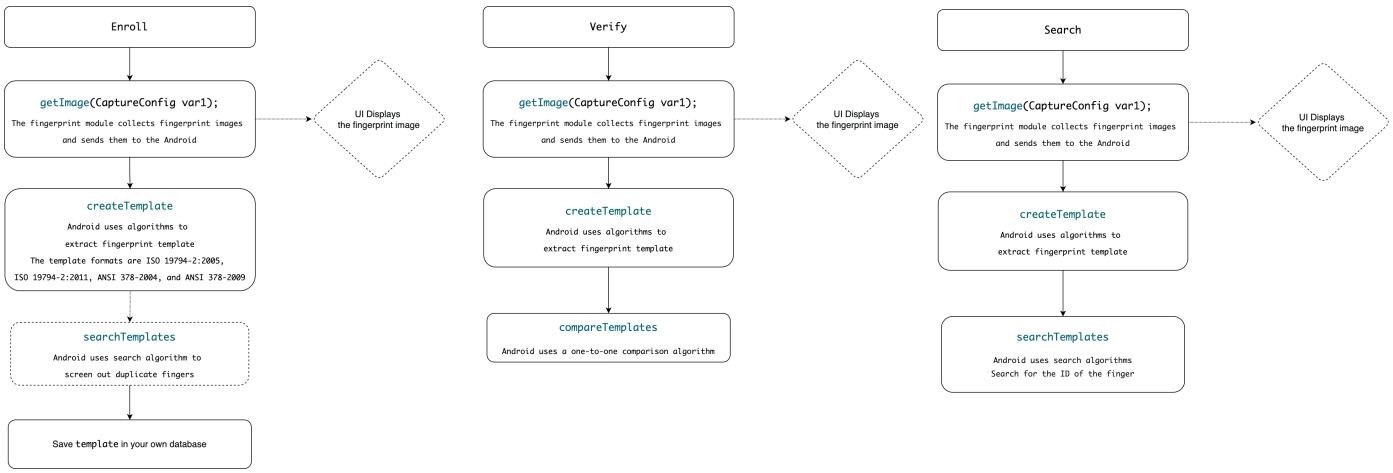
3.3 How to get high-quality fingerprints

The finger type is not conducive to obtaining high quality fingerprint	Fingerprint collected	How to get high-quality fingerprints
Wet finger		Need to remove the excessive moisture from your fingers and keep the sensor clean.
Dry finger		Use hand cream, wet wipes, blowing on his stiff fingers to moisturize.
Fingers have obvious scars		Use other fine fingers.
Finger peeling		Use other fine fingers.
Not enough pressure	Like the image of dry fingers	Should apply moderate but steady pressure, and stay 1 second, about the same, pressure as pressing a button.
Too much pressure	Like the image of wet fingers or torsion distortion	Should be relaxed and reduce the stress, apply moderate but steady pressure, and stay 1 second, about the same, pressure as pressing a button.

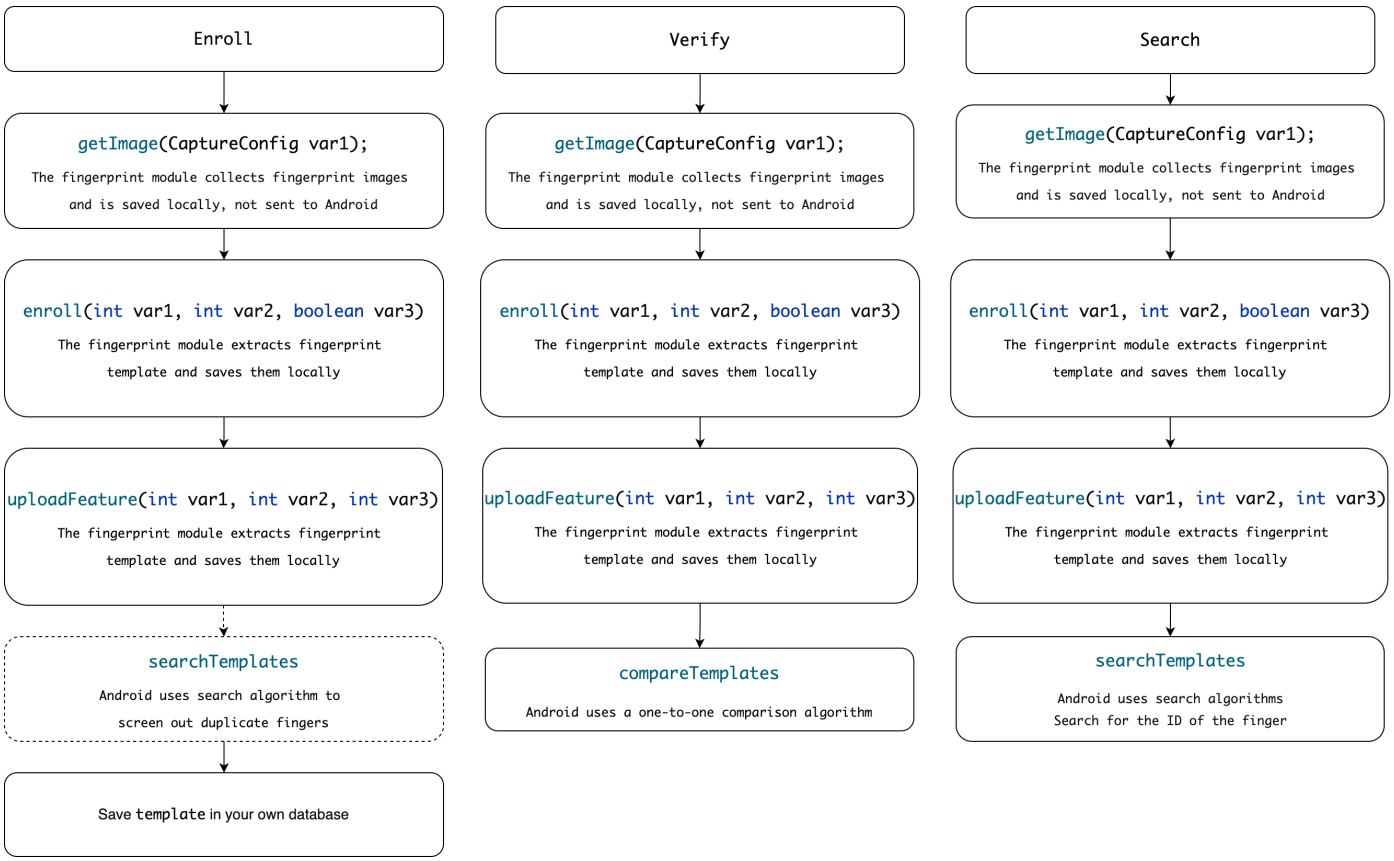
3.4 Development reference

Fingerprint template data is stored in your database.

3.4.1 Host (USB/SPI communication)

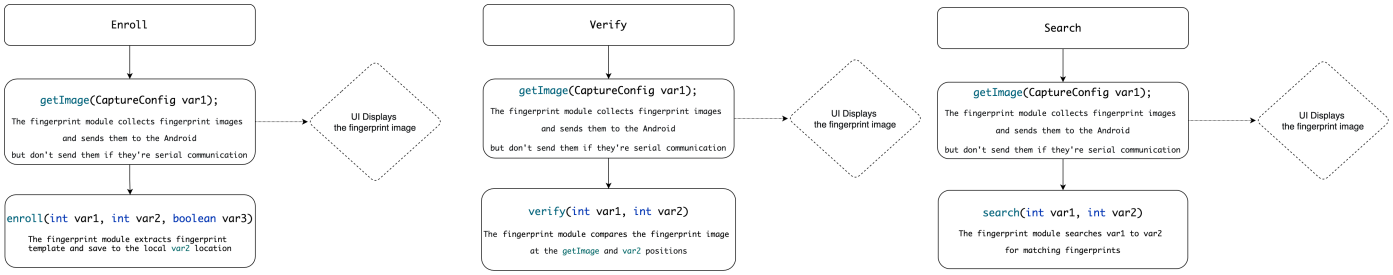


3.4.1 Host (Serial communication)



3.4.1 Module

The fingerprint template data is stored in the fingerprint module.


Enroll
getImage(CaptureConfig var1);
The fingerprint module collects fingerprint images
and sends them to the Android
but don't send them if they're serial communication
UI Displays
the fingerprint image
enroll(int var1, int var2, boolean var3)
The fingerprint module extracts fingerprint
template and save to the local var2 location
Verify
getImage(CaptureConfig var1);
The fingerprint module collects fingerprint images
and sends them to the Android
but don't send them if they're serial communication
UI Displays
the fingerprint image
verify(int var1, int var2)
The fingerprint module compares the fingerprint image
at the getImage and var2 positions
Search
getImage(CaptureConfig var1);
The fingerprint module collects fingerprint images
and sends them to the Android
but don't send them if they're serial communication
UI Displays
the fingerprint image
search(int var1, int var2)
The fingerprint module searches var1 to var2
for matching fingerprints